

Orchestrating a Reliable Data Product with Flyte

The focus of this practical assignment is to:

- Implement a production-grade orchestration layer (Flyte) on top of your lakehouse stack
- Apply reliability primitives:
 - Parameterization by logical time
 - Idempotent outputs (safe retries)
 - Timeouts + retries
 - Late data strategy (lookback window)
 - Backfills
- Operate like a data platform engineer:
 - Observe executions in Flyte UI (traceability)
 - Inspect Iceberg metadata (snapshots/files/partitions)
 - Use Trino UI stats as evidence for what happened (bytes/rows/time)

Scenario



You are asked to operationalize the data product you designed earlier for ShopNow, in the “From Data Pain to a Data Product” activity. If you did not complete this past activity and/or prefer to do the current activity applied to your practical project, feel free to do so. It can be easily adapted. We'll operationalize the creation of a data product:

- Bronze input: `bronze.web_events_raw` (append-only, duplicates possible, late arrivals up to 7 days)
- Target product: `gold.customer_daily_kpis` (stable daily features + KPIs; 1 row per customer per day)

Pain to fix (in production):

- There are duplicates after retries, leading to inflated KPIs
- Late events may arrive up to 7 days late
- Backfills are required (recompute past days safely)

Stack and Tools

- Trino (CLI or UI) to execute SQL transformations
- Iceberg catalog writing Parquet to MinIO
- MinIO Console to inspect objects (data/metadata layout)
- Flyte (Flyte UI + Flytekit) to orchestrate the pipeline

Expected deliverables at the end of the tutorial

1. A Flyte workflow (code) + a Launch Plan (or documented run command) that produces `gold.customer_daily_kpis`.
2. Evidence screenshots/snippets:
 - a. A “bad run” showing duplicate inflation after retry (before fix)
 - b. A “good run” showing idempotent behavior after fix
 - c. Flyte execution graph/details
 - d. Iceberg `$snapshots` history for the gold table and at least one `$files/$partitions` query
3. Short notes for discussion (5–10 lines) on
 - a. How you ensured idempotency
 - b. How you handle late data and backfills
 - c. What you’d change between dev vs prod

1. Create schemas for bronze/silver/gold

Create a new schema for this activity as follows.

```
CREATE SCHEMA IF NOT EXISTS iceberg.tead_flyte
WITH (location = 's3a://warehouse/tead_flyte/');

USE iceberg.tead_flyte;
SHOW TABLES;
```

Create the necessary tables. We keep everything in one Trino schema and use table prefixes `bronze_ / silver_ / gold_` to emulate layers:

- `bronze.web_events_raw` (append-only)
- `silver.web_events_clean` (deduped/normalized)
- `gold.customer_daily_kpis` (aggregated daily product)

```
USE iceberg.tead_flyte;

-- BRONZE (append-only, raw-ish)
DROP TABLE IF EXISTS bronze_web_events_raw;
CREATE TABLE bronze_web_events_raw (
  event_id VARCHAR,
  event_type VARCHAR, -- page_view, add_to_cart, checkout_started,
```

```
purchase, refund
customer_id VARCHAR, -- nullable
event_time TIMESTAMP,
order_id VARCHAR, -- nullable
order_total_eur DECIMAL(10,2), -- nullable
source VARCHAR, -- web, ios, android
ingest_time TIMESTAMP,
batch_id VARCHAR
)
WITH (
format = 'PARQUET',
partitioning = ARRAY['day(ingest_time)'] -- helps browsing + pruning
by ingest day
);

-- SILVER (deduped/normalized, one row per event_id)
DROP TABLE IF EXISTS silver_web_events_clean;
CREATE TABLE silver_web_events_clean (
event_id VARCHAR,
event_type VARCHAR,
customer_id VARCHAR,
event_time TIMESTAMP,
event_date DATE,
order_id VARCHAR,
order_total_eur DECIMAL(10,2),
source VARCHAR,
ingest_time TIMESTAMP,
batch_id VARCHAR
)
WITH (
format = 'PARQUET',
partitioning = ARRAY['event_date'] -- partition by logical event day
);

-- GOLD (data product)
DROP TABLE IF EXISTS gold_customer_daily_kpis;
CREATE TABLE gold_customer_daily_kpis (
day DATE,
customer_id VARCHAR,
page_views BIGINT,
add_to_cart BIGINT,
checkout_started BIGINT,
purchases BIGINT,
orders BIGINT,
revenue_eur DECIMAL(18,2),
active BOOLEAN,
last_event_time TIMESTAMP,
produced_at TIMESTAMP
)
WITH (
format = 'PARQUET',
partitioning = ARRAY['day'] -- 1 partition per day
);
```

2. Seed some bronze data

This generates a “realistic enough” dataset with:

- event distribution across types
- null customers
- late arrivals up to 7 days (ingest_time delayed vs event_time)
- duplicates for a subset of events (re-sends)

```
USE iceberg.tead_flyte;

-- Generate base events
WITH base AS (
  SELECT x AS n
  FROM UNNEST(sequence(1, 200000)) t(x) -- adjust volume if needed
),
events_base AS (
  SELECT
    CAST(n AS VARCHAR) AS event_id,
    CASE
      WHEN (n % 100) < 70 THEN 'page_view'
      WHEN (n % 100) < 90 THEN 'add_to_cart'
      WHEN (n % 100) < 95 THEN 'checkout_started'
      WHEN (n % 100) < 99 THEN 'purchase'
      ELSE 'refund'
    END AS event_type,

    CASE WHEN (n % 20) = 0 THEN NULL ELSE CAST(1 + (n % 50000) AS VARCHAR)
    END AS customer_id,

    -- event_time spread over ~30 days ending at a fixed anchor
    date_add('minute', -(n % 43200), TIMESTAMP '2026-02-26 00:00:00') AS
    event_time,

    CASE
      WHEN (n % 100) >= 99 THEN CONCAT('o', CAST(n - 1 AS VARCHAR)) --
      refund references previous order sometimes
      WHEN (n % 100) >= 96 THEN CONCAT('o', CAST(n AS VARCHAR)) -- purchase
      ELSE NULL
    END AS order_id,

    CASE
      WHEN (n % 100) >= 99 THEN -CAST((10 + (n % 200)) AS DECIMAL(10,2)) --
      refund negative
      WHEN (n % 100) >= 96 THEN CAST((10 + (n % 200)) AS DECIMAL(10,2))
      ELSE NULL
    END AS order_total_eur,

    element_at(ARRAY['web','ios','android'], 1 + (n % 3)) AS source,

    -- ingest_time delayed by 0..7 days (late arrivals)
    date_add('second', (n % 3600),
```

```
date_add('day', (n % 8),
date_add('minute', -(n % 43200), TIMESTAMP '2026-02-26 00:00:00')
)
) AS ingest_time,

CONCAT('b', CAST(1 + (n % 50) AS VARCHAR)) AS batch_id,

n
FROM base
)

-- Insert base rows
INSERT INTO bronze_web_events_raw
SELECT
event_id, event_type, customer_id, event_time, order_id,
order_total_eur, source, ingest_time, batch_id
FROM events_base

UNION ALL

-- Create duplicates (re-sends) for ~2% of events
SELECT
event_id,
event_type,
customer_id,
event_time,
order_id,
order_total_eur,
source,
date_add('minute', 10, ingest_time) AS ingest_time, -- resent later
CONCAT(batch_id, '_retry') AS batch_id
FROM events_base
WHERE (n % 50) = 0;
```

Now confirm that there are duplicates and late data:

```
USE iceberg.tead_flyte;

-- Total rows
SELECT count(*) AS n_rows FROM bronze_web_events_raw;

-- How many duplicated event_ids?
SELECT count(*) AS duplicated_event_ids
FROM (
  SELECT event_id
  FROM bronze_web_events_raw
  GROUP BY 1
  HAVING count(*) > 1
);

-- Late arrivals: max delay (in days) between event_time and ingest_time
SELECT max(date_diff('day', event_time, ingest_time)) AS max_delay_days
FROM bronze_web_events_raw;
```

4. (A bad) Flyte workflow

This version appends into gold and then fails, so Flyte retries the same task → duplicates appear (the “pain” from the spec). Put the attached `shopnow_flyte_v0.py` script in the project folder.

Next, run the workflow locally to reproduce duplicates after retries.

```
export TRINO_HOST=localhost
export TRINO_PORT=8080
export TRINO_USER=tead
export TRINO_CATALOG=iceberg
export TRINO_SCHEMA=tead_flyte

pyflyte run shopnow_flyte.py shopnow_kpis_v0 --logical_date 2026-02-25
--lookback_days 0 --fail_after_write True
```

You should see the gold task run, write, then failing. Flyte retries, it writes again, which leads to duplicates. Then validation fails.

Now inspect Iceberg snapshots, where you should see multiple commits for gold:

```
SELECT committed_at, snapshot_id, operation
FROM "gold_customer_daily_kpis$snapshots"
ORDER BY committed_at DESC;
```

5. Fix it

Now we replace the unsafe append with a MERGE keyed on (day, customer_id). This makes the gold step safe to retry. This is done in the script `shopnow_flyte_v1.py`.

Now let's run it multiple times for the same day:

```
pyflyte run shopnow_flyte.py shopnow_kpis_v1 --logical_date 2026-02-25
--lookback_days 0
pyflyte run shopnow_flyte.py shopnow_kpis_v1 --logical_date 2026-02-25
--lookback_days 0
```

And confirm idempotency:

```
SELECT count(*) FROM (
  SELECT day, customer_id, count(*) c
  FROM iceberg.tead_flyte.gold_customer_daily_kpis
  WHERE day = DATE '2026-02-25'
  GROUP BY 1,2
  HAVING count(*) > 1
);
```

6. Demonstrate late data handling

We'll Insert "late arrivals" into Bronze. These events happened on 2026-02-22 but arrive later (ingest on 2026-02-26).

```
USE iceberg.tead_flyte;

INSERT INTO bronze_web_events_raw
SELECT
  CONCAT('late_', CAST(x AS VARCHAR)) AS event_id,
  'purchase' AS event_type,
  CAST(1000 + x AS VARCHAR) AS customer_id,
  TIMESTAMP '2026-02-22 12:00:00' AS event_time,
  CONCAT('o_late_', CAST(x AS VARCHAR)) AS order_id,
  CAST(50.00 AS DECIMAL(10,2)) AS order_total_eur,
  'web' AS source,
  TIMESTAMP '2026-02-26 09:00:00' AS ingest_time,
  'late_batch' AS batch_id
FROM UNNEST(sequence(1, 50)) t(x);
```

Run the pipeline with lookback so it repairs the past:

```
pyflyte run shopnow_flyte.py shopnow_kpis_v1 --logical_date 2026-02-25
--lookback_days 7
```

Now confirm that 2026-02-22 gold changed (revenue increased) but still no duplicates:

```
SELECT day, sum(revenue_eur) AS revenue
FROM iceberg.tead_flyte.gold_customer_daily_kpis
WHERE day = DATE '2026-02-22'
GROUP BY 1;

SELECT count(*) FROM (
  SELECT day, customer_id, count(*) c
  FROM iceberg.tead_flyte.gold_customer_daily_kpis
  WHERE day = DATE '2026-02-22'
  GROUP BY 1,2
  HAVING count(*) > 1
);
```

7. Backfill based on date range

Copy the script `backfill.py` to your project and run it:

```
python backfill.py --start 2026-02-10 --end 2026-02-25 --lookback_days
7
```

In each run, this script:

- Upserts silver for the window covering D plus lookback
- MERGEs gold for the days in that window

- Validates constraints

After the backfill:

- Gold should have complete partitions for the entire date range.
- Rerunning any of those dates should yield no duplicates and consistent totals.
- Iceberg snapshots should show a sequence of commits that correspond to those runs.
- In Flyte UI you can see a batch of executions and inspect one run end-to-end.

8. Evidence checkpoints (what to look at)

We shouldn't just run a pipeline: we should learn how to prove what happened, debug when it doesn't, and reason about correctness and performance using the platform's metadata and execution traces. This mirrors real production work. Our sources of evidence are:

- Iceberg metadata checks (what changed in the lakehouse)
 - Snapshots: show that runs create commits, and compare operation and timing.
 - Files: show how many files were written, file sizes, whether "small files" appear.
 - Partitions: confirm which days/partitions were affected and how big they are.
- Trino query evidence (what the engine actually did). For the silver MERGE and gold MERGE queries, we can capture
 - Scanned bytes/rows
 - Stages / time breakdown
 - Whether partition pruning is happening (less scan) vs full-table scans
- Flyte execution evidence (how the control plane behaved). From Flyte UI, we capture:
 - The execution graph
 - Inputs (logical_date, lookback_days)
 - Task retries/timeouts behavior (especially v0 vs v1)
 - Logs for the SQL tasks

Iceberg Metadata

```
USE iceberg.tead_flyte;

SELECT committed_at, snapshot_id, operation
FROM "gold_customer_daily_kpis$snapshots"
ORDER BY committed_at DESC;

SELECT * FROM "gold_customer_daily_kpis$partitions" ORDER BY
total_size DESC;

SELECT file_path, record_count, file_size_in_bytes
```



```
FROM "gold_customer_daily_kpis$files"  
ORDER BY file_size_in_bytes ASC  
LIMIT 20;
```

Trino UI query stats

Look at the logs and, For the silver MERGE and gold MERGE queries, analyze:

- Processed rows/bytes
- Stage breakdown
- Any obvious skew/shuffle symptoms

Trino UI query stats

Analyze:

- Execution inputs: logical_date, lookback_days
- Node logs: show SQL statement and/or Trino query id (you can print it if you want)
- Confirm retries happened (in v0) and don't break correctness (in v1)